

AGRIAI 2.0

INTELLIGENT FARMING ASSISTANT FOR GHANA

Volume II. How the AI Actually Works

Gemini, Cloudflare, local knowledge, and why answers still arrive

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Computer Science, University of Ghana

Founder, designer, and sole engineer of AgriAI

Repository: ThaddeusHackz/agriai-v2

Document date: 18 August 2026

Classification: Internal forensic architecture pack

This document explains how AgriAI was built, which source files make the AI work, and how the public and admin APIs stay reliable when a provider is missing. Every claim below is taken from the live codebase, not from marketing copy.

1. The question this volume answers

A reader who is not a machine learning engineer still needs to understand why AgriAI can speak like an agronomist. There is no custom trained crop model sitting in this repo. The intelligence is a carefully ordered pipeline: system prompts written for Ghana, live model calls, optional web grounding, and a local knowledge base that never lets the product go silent.

2. The three layer brain

Every chat request walks the same ladder. Layer 1 is Google Gemini (primary). Layer 2 is Cloudflare Workers AI Llama 3.3 70B instruct (automatic fallback). Layer 3 is `localAnswer()` in `lib/ai.ts`, which searches the admin editable knowledge base and then regex curated Ghana answers for maize, cassava, cocoa, tomato, armyworm, loans, and weather. The product never returns an empty bubble because of a missing key.

3. File that makes Gemini possible: `lib/ai.ts`

`getGemini()` reads `geminiApiKey()` from `lib/env.ts`. Accepted env names are `GEMINI_API_KEY`, `GOOGLE_API_KEY`, `GOOGLE_GENERATIVE_AI_API_KEY`, and `GOOGLE_GENAI_API_KEY`. The client is cached and rebuilt if the key changes at runtime. `geminiConfigured()` is the boolean the chat route uses before it even tries a live call.

4. Model waterfall (why one failed model does not kill chat)

CHAT_MODELS is an ordered list: gemini-2.5-flash, gemini-2.5-flash-lite, gemini-2.0-flash, gemini-2.0-flash-001, gemini-flash-latest, gemini-1.5-flash, gemini-1.5-flash-latest. uniqueModels(preferred) puts the admin chosen model first, then the rest, without duplicates. geminiGenerateText and geminiGenerateStream loop that list. If a model returns empty text or throws, the next model is tried. Gemini 2.5 models receive thinkingBudget 0 so the Flash thinking tax does not eat the token budget.

5. Streaming is how the UI feels instant

geminiGenerateStream calls client.models.generateContentStream. Each chunk.text is handed to onDelta. The chat route wraps that in a ReadableStream that writes Server Sent Events: event delta with a JSON text fragment, then event done with the full text, demo flag, sources, searchDemo, and provider (gemini, cloudflare, or local). The browser Chat component paints tokens as they arrive. If Gemini fails mid way, the route clears fullText and tries Cloudflare, then local.

6. System prompts: the Ghana personality

Prompts live in the database settings.chat object, seeded in lib/db.ts, editable in the admin AI tab. Three modes exist.

- standard (systemPrompt): AgriAI as a practical Ghana farming assistant built by Thaddeus Tagoe. Crops named: maize, cocoa, cassava, yam, plantain, rice, tomatoes, peppers, groundnuts. Simple language a rural farmer can use. Cite [1] [2] when search context is present.
- expert (expertPrompt): agronomist voice. NPK ratios, rates, densities, disease life cycles, IPM, GHS and kg/ha numbers.
- agent (agentPrompt): research agent. Break the question, use search results, return summary, numbered points, citations, sections.

languageInstruction(language) from lib/languages.ts is appended so replies can be English, Twi, Ga, Ewe, Hausa, or French. If web search ran, CURRENT WEB SEARCH RESULTS are appended to the system string. The model is told to cite [1], [2]. That is how grounding works without a vector database.

7. Conversation memory without a vector store

The client sends history: last user and assistant turns. The route keeps the last ten valid messages, clips each to 4000 characters, and sends the last eight to Gemini as role user or role model parts. After the answer, mutate() appends both the farmer message and the assistant message to a chat keyed by sessionId. Admin Messages tab reads that same document. There is no Pinecone, no embeddings file, no RAG index. Recall is short context plus the knowledge base.

8. Web search grounding: lib/search.ts

When the farmer toggles webSearch, the chat route calls searchWeb(message + " Ghana 2026"). If TAVILY_API_KEY exists, POST https://api.tavily.com/search with advanced depth, include_answer, max_results 6. On missing key or Tavily error, keylessSearch hits Wikipedia OpenSearch and DuckDuckGo Instant Answer. contextBlock() turns the result into a numbered text block the LLM can cite. Sources travel back on the done event so the UI can show links.

9. Vision: crop disease detection

`detectCropDisease(imageInput, language)` in `lib/ai.ts` is the whole vision product. `parseImageData` accepts a data URL or raw base64. The prompt forces STRICT JSON: detected, confidence 0-100, description, treatment array. Temperature is 0.2 so the model stays conservative. The same CHAT_MODELS waterfall is used, preferring `settings.chat.visionModel`. JSON is extracted with a brace regex and parsed. If no key exists, `FALLBACK_DISEASE` is returned with `demo true` so the UI still teaches the farmer what a live result looks like. `/api/vision` is a thin validator: language whitelist, size cap, then `detectCropDisease`.

10. Voice in and voice out

Primary voice input is the browser SpeechRecognition API inside `Chat.tsx`. Browsers without it POST audio FormData to `/api/transcribe`. That route base64 encodes the file and asks Gemini 2.5 Flash to transcribe verbatim with temperature 0. One key therefore powers chat, vision, and speech to text. Voice output is `/api/tts`. It sends up to 1500 characters to ElevenLabs `eleven_multilingual_v2` with voice Rachel for English and French and a multilingual default for Ghanaian languages. The route returns `audio/mpeg`. If `ELEVENLABS_API_KEY` is missing, status 501 tells the UI to hide or disable speak.

11. Image generation: AgriAI Studio

`POST /api/generate` first requires Cloudflare or Gemini to be configured. The prompt must mention a crop or farm word from `SAFE_SUBJECTS` (maize, cocoa, cassava, yam, and many others). The prompt is rewritten with photorealistic agricultural photography language. Cloudflare Flux-1-schnell runs first (4 steps). Gemini native image models are the second try: `gemini-2.5-flash-image`, then two 2.0 preview image models. The response is a PNG or JPEG data URL. This is how Crop Visualizer works without a separate GPU box.

12. Cloudflare as the insurance policy: lib/cloudflare.ts

`cloudflareConfigured()` needs both `CLOUDFLARE_API_KEY` and `CLOUDFLARE_ACCOUNT_ID`. Chat model is `@cf/meta/llama-3.3-70b-instruct-fp8-fast`. Image model is `@cf/black-forest-labs/flux-1-schnell`. Calls are POST to `https://api.cloudflare.com/client/v4/accounts/{id}/ai/run/{model}` with a Bearer token and a 30 second abort. Failures throw so the caller can step down the ladder.

13. Local knowledge: the AI that works on an airplane

`localAnswer(question, language, mode)` first scans `db.knowledge` for keyword hits or a prefix of the stored question. Then `FALLBACK_ANSWERS` regexes fire. Then a generic AgriAI help card is returned, naming crops, pests, soil, prices, and weather. Admin staff can add knowledge rows without a deploy. Those rows become live fallbacks immediately because `getDB()` is in process memory.

14. What was not built (honest forensic note)

There is no fine tuned Ghana crop weights file. There is no on device TFLite model. Disease detection accuracy advertised in seed stats is a product metric, not a confusion matrix checked into this repo. The AI works because Thaddeus Tagoe wired production APIs, Ghana specific prompts, search citations, and a fail closed local brain into one typed module. That engineering is the invention.

15. End to end farmer story

A farmer in Kumasi types in Twi, enables web search, and asks about cocoa. Chat.tsx POSTs to /api/chat. Analytics increment. Tavily (or Wikipedia) returns sources. Gemini streams an expert or standard answer in Twi with [1] markers. The done event carries provider gemini. The farmer taps speak. /api/tts returns MP3. If Gemini is down the same second, Llama answers. If Cloudflare is also down, the cocoa knowledge article still teaches COCOBOD grades and licensed buying companies. That chain is why the AI feels reliable.